EcoShare

Smart Food Waste Management System

Submitted to :- Mahipal Singh Papola , Deepak Kumar

TEAM – MEMBERS

RONIT SAHA – **12400705**

SOMNATH BHASKAR-**12401921**

PRATEEK KUMAR PAL**12418501**

KONDIPATI HARSHA VARDHAN-**12409535**

BEEDALA PAVAN KUMAR REDDY-**12406337**

Live website link:-

https://eco-share-smart-food-waste-manageme.vercel.app/

2. Abstract

Food waste is an escalating and critical global challenge, with approximately one-third of all food produced for human consumption globally being lost or wasted every single year. This staggering inefficiency in the global food supply chain occurs at every stage—from agricultural production and processing to retail and consumer levels. Conversely, while mountains of edible food are deposited into landfills, millions of people worldwide continue to suffer from food insecurity, malnutrition, and hunger.

The dichotomy between massive food surplus and widespread food scarcity presents a unique opportunity for technological intervention. **EcoShare** is conceived as a comprehensive, full-stack software platform designed specifically to bridge this gap. By acting as a digital conduit, EcoShare connects potential food Donors—primarily commercial entities such as restaurants, banquet halls, and institutional cafeterias—with Receivers, which include non-governmental organizations (NGOs), student bodies, and community shelters.

The core philosophy of the system revolves around intelligent redistribution, leveraging real-time tracking, geolocation services, and automated claiming protocols to mitigate food waste precisely at its source. By instantly broadcasting available surplus, the platform drastically reduces the time window between food generation and consumption, which is critical for perishable cooked goods.

From a technical standpoint, EcoShare is built upon the modern and highly robust MERN stack (MongoDB, Express.js, React.js, and Node.js), augmented with TypeScript for enterprise-grade type safety. The application architecture is further enhanced with robust User Interface libraries such as Tailwind CSS for responsive design, Framer Motion for fluid micro-interactions, and Three.js for engaging visual representations. Consequently, EcoShare offers a highly interactive, scalable, and secure environment, ensuring that surplus food is efficiently and equitably allocated before it perishes, driving both social equity and environmental sustainability.

3. Introduction

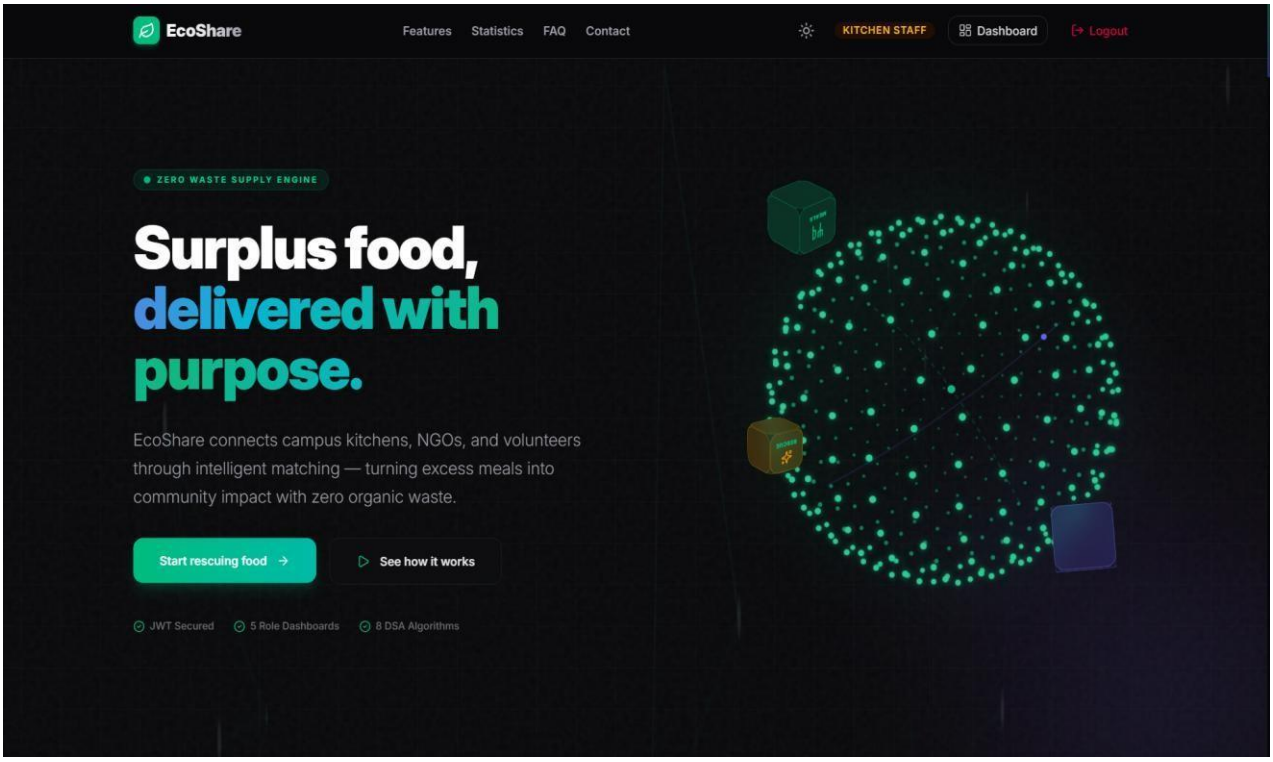
3.1 Problem Statement

The modern urban ecosystem is characterized by rapid consumption and high-volume food production. In these urban areas, commercial food establishments such as restaurants, corporate cafeterias, bakeries, and large-scale banquet halls frequently generate a significant amount of surplus, perfectly edible food at the end of their daily operational cycles.

Despite the inherent value of this surplus, the primary obstacle to its redistribution is logistical friction. Due to the lack of a centralized, real-time, reliable, and swift communication medium between these potential donors and the localized food-insecure populations—such as struggling university students, low-income families, or charitable NGOs—this perfectly good food is overwhelmingly discarded into municipal solid waste streams.

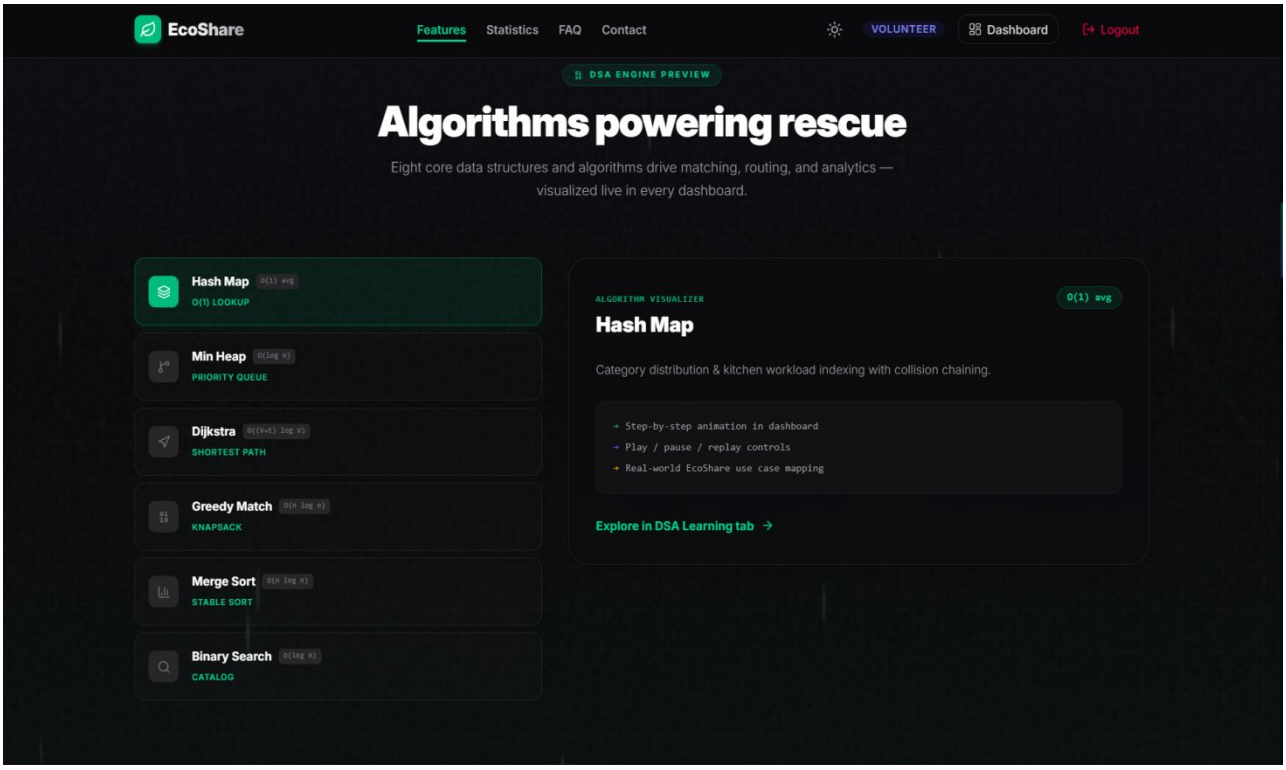
The consequences of this logistical failure are twofold. Environmentally, organic waste deposited in landfills undergoes anaerobic decomposition, resulting in the massive emission of methane gas. Methane is an aggressive greenhouse gas with a global warming potential over 28 times greater than that of carbon dioxide over a 100-year period, making food waste a leading contributor to anthropogenic climate change.

Socially and ethically, the disposal of nutritious food is a grave moral failure in a world where hunger remains rampant. The inability to rapidly match available supply with desperate demand highlights a severe inefficiency in urban resource management. Therefore, there is a critical and immediate need for a digital infrastructure capable of bypassing traditional, slow-moving donation channels to facilitate the instantaneous transfer of highly perishable food items from those who have too much, to those who have too little.



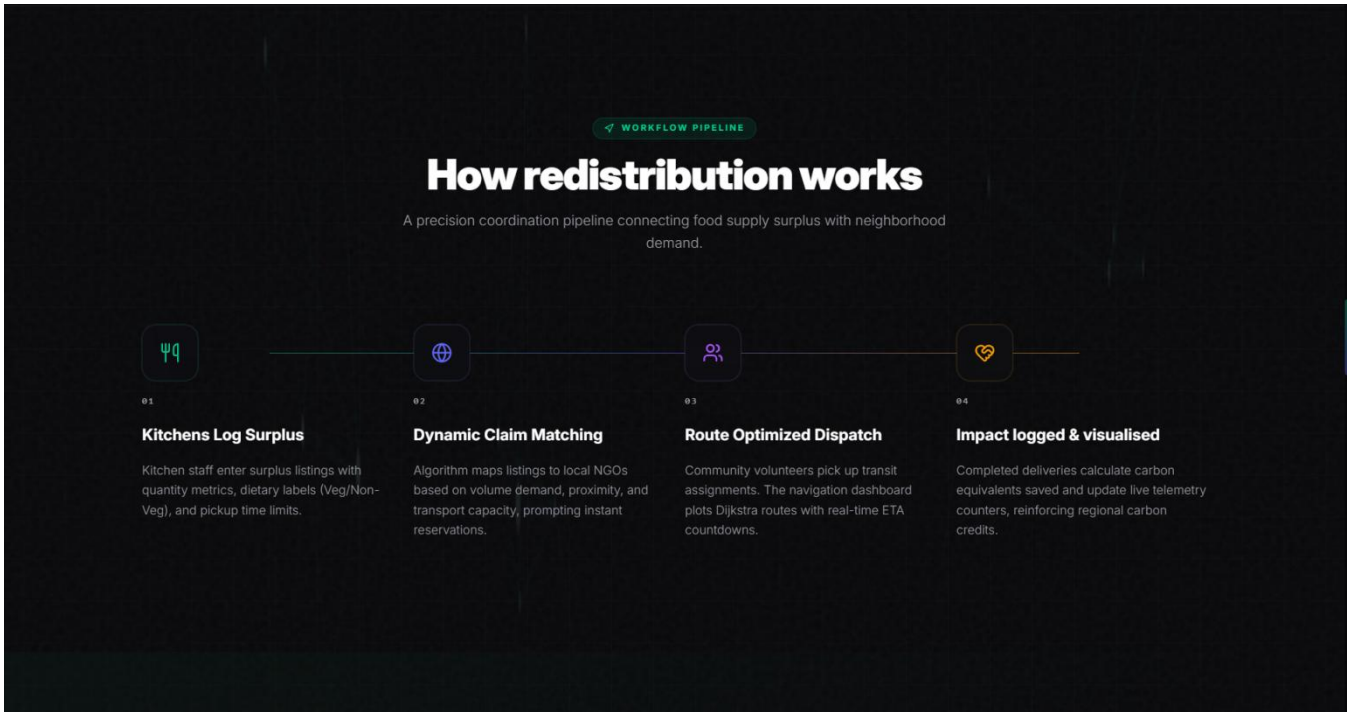
##Photo 1 (Homepage):

EcoShare's intelligent platform connects surplus food from campuses to NGOs, minimizing waste and maximizing community impact.



##Photo 2 (DSA Features):

Interactive DSA visualizations demonstrate how core algorithms power EcoShare's fast and efficient food redistribution system.



##Photo 3 (Workflow):

A step-by-step workflow shows how surplus food is matched, routed, and delivered efficiently using intelligent algorithms.

3.2 Objectives

The development of the EcoShare project is guided by a set of primary objectives aimed at maximizing both social impact and technological efficacy:

- **Reduce Food Waste at the Source:** Provide a real-time, frictionless platform for commercial food businesses to list their end-of-day surplus food instantaneously, transforming potential waste into a recoverable asset.

- **Alleviate Localized Hunger:** Empower NGOs, shelters, and verified individuals in need with a tool to locate, reserve, and claim nearby available food seamlessly, breaking down barriers to food access .

- **Data-Driven Impact Tracking:** Offer comprehensive analytics dashboards for donors. By quantifying the weight of food donated and calculating the corresponding environmental metrics (e.g., carbon emissions prevented), the platform incentivizes continuous participation through Corporate Social Responsibility (CSR) reporting.

- **Facilitate Seamless Communication:** Eliminate the traditional friction between the donor and receiver via automated push alerts, real-time status updates, and digital claiming mechanisms, entirely removing the need for manual phone calls or emails.

3.3 Why EcoShare? (Motivation)

1. Problem Statement

Traditional methods of charitable food donation rely heavily on manual coordination, phone calls, fixed schedules, and delayed transportation. While these approaches may be effective for non-perishable food items, they are not suitable for freshly prepared surplus food, which has a very limited safe consumption period. As a result, large quantities of edible food are wasted every day despite many people facing food insecurity.

2. Inspiration Behind EcoShare

The inspiration for **EcoShare** originated from a real-life observation in the college hostel. During a conversation, the hostel warden highlighted that many students often serve themselves more food than they can consume, resulting in a significant amount of perfectly edible food being discarded every day. This simple yet impactful observation made the team realize that food wastage was not just a global issue but also a problem occurring within the campus community.

3. Project Motivation

When the opportunity arose to develop a **Data Structures and Algorithms (DSA)** project, the team saw it as an ideal opportunity to design a solution that could address a real-world challenge while demonstrating technical and problem-solving skills. Instead of creating a purely academic application, **EcoShare** was developed as a web-based platform that connects food donors with nearby NGOs and volunteers, enabling the timely redistribution of surplus food before it expires.

4. Proposed Solution

EcoShare digitizes and automates the entire food redistribution process using modern web technologies. By integrating features such as geolocation services, real-time database

updates, and secure role-based authentication, the system enables restaurants, hostels, and event organizers to post surplus food within seconds, while nearby NGOs can instantly discover and claim available donations. This significantly reduces response time, minimizes food wastage, and demonstrates how data structures and algorithms can be effectively applied to solve meaningful real-life problems.

Core Modules and Workings

3.4 User Authentication & Role Management Module

The foundation of EcoShare is built upon strict identity and role management. Because the system serves drastically different user bases (restaurants posting food vs. NGOs claiming food), the user experience must be entirely customized based on authorization levels.

1. **Registration Workflow:**

When a user signs up, they must explicitly declare their organizational type by selecting a role ('Donor' or 'Receiver'). The backend receives this data, validates the email format, and checks against the database to ensure the email is unique. The raw password string is then passed through the `bcrypt` hashing algorithm (with a salt factor of 10) to generate a secure hash, which is finally committed to the MongoDB database

2. **Login and Session Management Flow:**

Upon attempting to log in, the backend retrieves the user by email and utilizes `bcrypt.compare()` to verify the password. If successful, the server utilizes the `jsonwebtoken` library to sign a secure JWT payload containing the user's `_id` and `role`. This token is transmitted back to the client application, which stores it securely in memory and attaches it as a Bearer Token in the HTTP headers for

all

future API requests.

3. **Role-Based UI Routing:**

The React frontend implements a Higher-Order Component (HOC) called `ProtectedRoute`. This component intercepts route changes; it decodes the local JWT and checks the user's role. If a Donor attempts to navigate to the Receiver's feed via the URL bar, the router intercepts the action and redirects them to an 'Unauthorized' page, completely isolating the user experience

3.5 Donor Module (Restaurants/Kitchens)

The Donor module is optimized for extreme speed and minimal friction, acknowledging that restaurant staff at closing time are busy and will abandon complex forms.

1. **Surplus Posting System:** The primary interface for Donors is the 'Post Donation' form. It requires minimal data entry: a description of the food (e.g., "3 Trays of Lasagna"), the estimated quantity (servings or kilograms), and a critical 'Valid Until' timestamp. Upon submission, the frontend captures the device's current geolocation (via browser API, with user consent) and bundles it with the form data. This guarantees that the food listing has an exact, verified origin point.

2. **Real-time Impact Analytics:** To encourage continuous engagement, the donor's dashboard serves as an interactive report card. By querying the backend for all historical food documents tied to their `donorId` where status is 'Completed', the frontend calculates the total weight of food diverted from the trash .

Using industry-standard conversion metrics (e.g., 1 kg of food waste = 2.5 kg of CO2 equivalent emissions), the Recharts library dynamically renders charts displaying the restaurant's environmental impact over the last 30, 60, or 90 days. This data is highly valuable for the commercial entity's marketing and Corporate Social Responsibility (CSR) portfolios.

3.6 Receiver Module (NGOs/Students)

The Receiver module prioritizes real-time data accuracy, ensuring that individuals in need have the most up-to-date view of available resources in their immediate vicinity.

1. **Dynamic Real-Time Feed:** Upon logging in, Receivers are presented with a live feed of food cards. The React application polls the backend API `/api/food?status=Available`, which queries the MongoDB database. The backend uses the receiver's current coordinates to filter out food located further than a specified radius (e.g., 15 kilometers), sorting the remaining results by proximity.

2. **The Automated Claiming System:** When a receiver identifies suitable food, they click the prominently displayed "Claim" button. This triggers an asynchronous `PUT` request to the backend. The backend immediately attempts to alter the document's status. If successful, the food card instantly disappears from the public feed (preventing others from attempting to claim it) and moves into the Receiver's 'Active Claims' tab, revealing the exact street address and pickup instructions of the Donor.

3. **Visual Feedback:** To provide positive reinforcement upon a successful claim, the frontend triggers a micro-animation utilizing `canvas-confetti` and Framer Motion, celebrating the successful rescue of the food item.

3.7 Admin Module

The Administrator module serves as the command center for the platform, ensuring the integrity, safety, and health of the EcoShare ecosystem. It is restricted to high-level staff.

1. **System Monitoring and Platform Health:** Admins have access to a bird's-eye view of all platform operations. A dedicated analytics panel aggregates data across the entire database, displaying metrics such as:

- Total number of registered users, segmented by Donors and Receivers.
- Total volume of food successfully redistributed since platform inception.
- Average time taken between a food item being posted and being claimed.

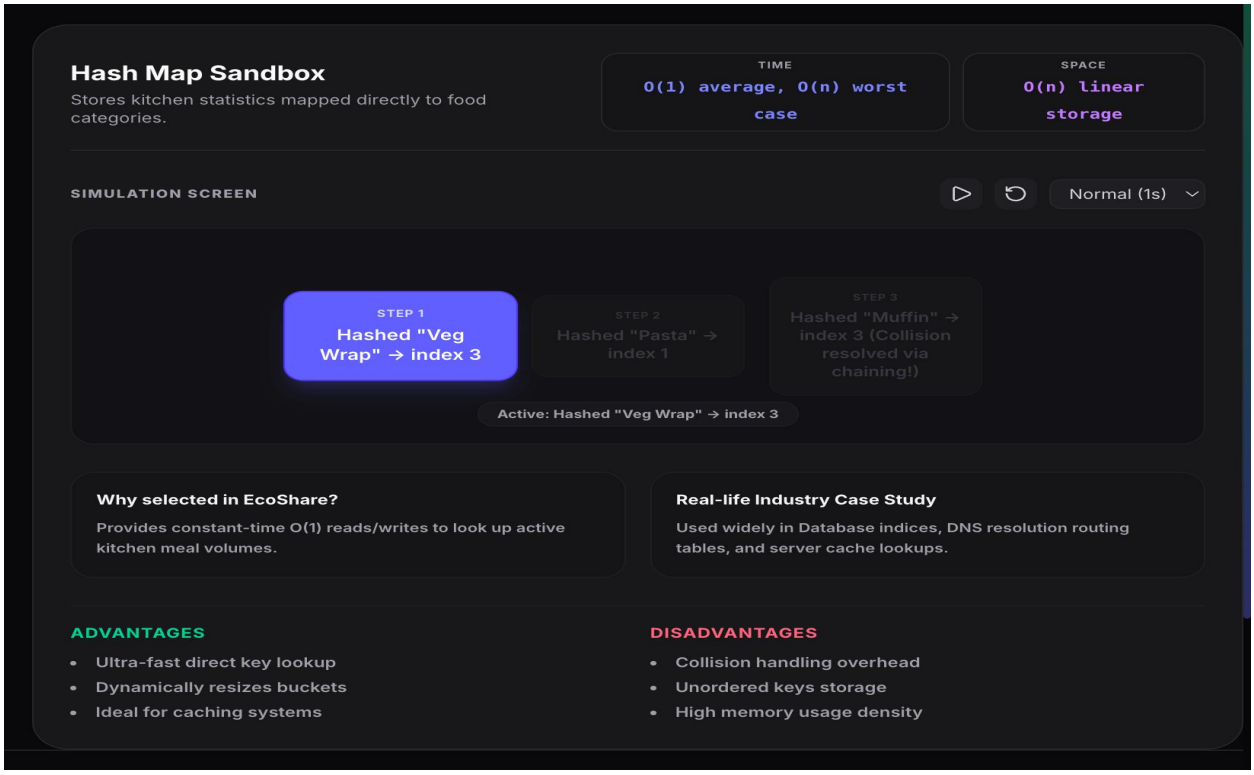
2. **User Verification & Dispute Resolution:** To prevent misuse of the platform (e.g., a bad actor creating a fake NGO account to claim food for personal commercial use), the admin module includes a verification queue. New Receiver accounts can be placed in a 'Pending' state until an admin manually verifies their non-profit credentials. Furthermore, admins possess the authorization to ban users, delete inappropriate listings, and manually override the status of donations in the event of a dispute or a no-show during pickup.

4. Algorithms and Logic Deep Dive

The technical backbone of EcoShare relies on several sophisticated algorithms to ensure data integrity, spatial accuracy, and ironclad security. The following sections dissect these computational methodologies.

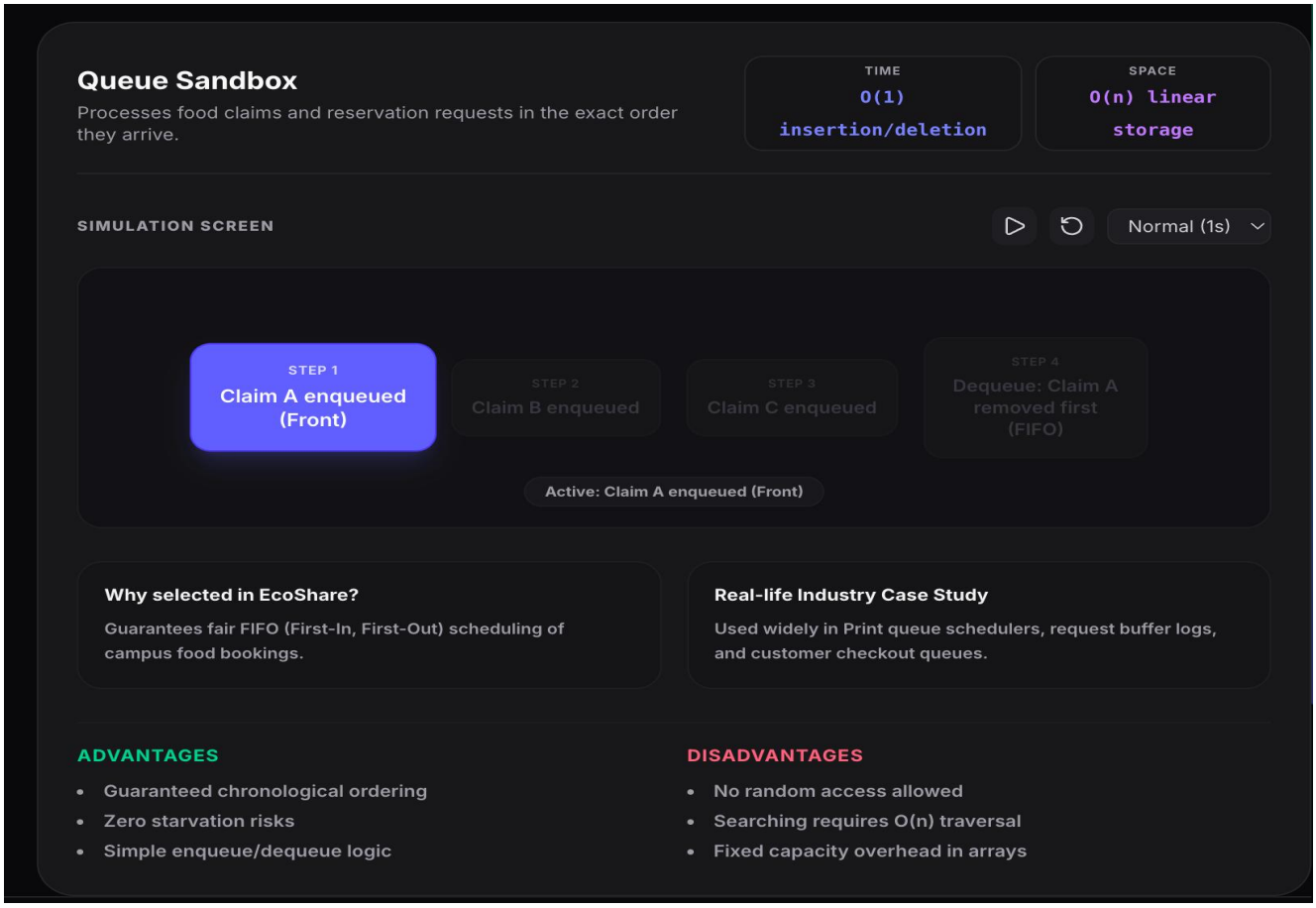
1. Hash Map:

- A Hash Map stores data as key-value pairs using a hash function for fast access.
- Time: O(1) average, O(n) worst case
- Space: O(n)
- Use in EcoShare: Quickly stores and retrieves food category data (stock, demand, etc.).
- Pros: Fast lookup, insert, and update.
- Cons: Collisions can reduce performance.



2. ***Queue:***

- A Queue is a linear data structure that follows the FIFO (First-In, First-Out) principle, where the first element added is the first one removed.
- Time: O(1) enqueue/dequeue
- Space: O(n)
- Use in EcoShare: Processes food claims and reservations in the order they are received.
- Pros: Fair processing, simple implementation.
- Cons: No random access, searching takes O(n).



3. ***Priority Queue:***

- A Priority Queue processes elements based on priority instead of arrival order, with higher-priority items served first.
- Time: O(log n) enqueue, O(1) peek
- Space: O(n)
- Use in EcoShare: Prioritizes highly perishable food for faster pickup and delivery.
- Pros: Efficient priority-based processing.
- Cons: Lower-priority items may wait longer.

Priority Queue Sandbox

Matches volunteers to active NGO dispatches prioritizing food perishability.

TIME	SPACE
`O(log n) enqueue, O(1) peek`	`O(n) linear storage`

SIMULATION SCREEN

Normal (1s)

STEP 1
Rider assigned route (Perishable index 10)

STEP 2
New route logged (Perishable index 30)

STEP 3
Sort priority: Index 30 moved to front of queue

Active: Rider assigned route (Perishable index 10)

Why selected in EcoShare?

Ensures hot/highly perishable meals are picked up and delivered before shelf-life expires.

Real-life Industry Case Study

Used widely in Hospital emergency room scheduling, routers routing voice traffic, and OS task management.

ADVANTAGES

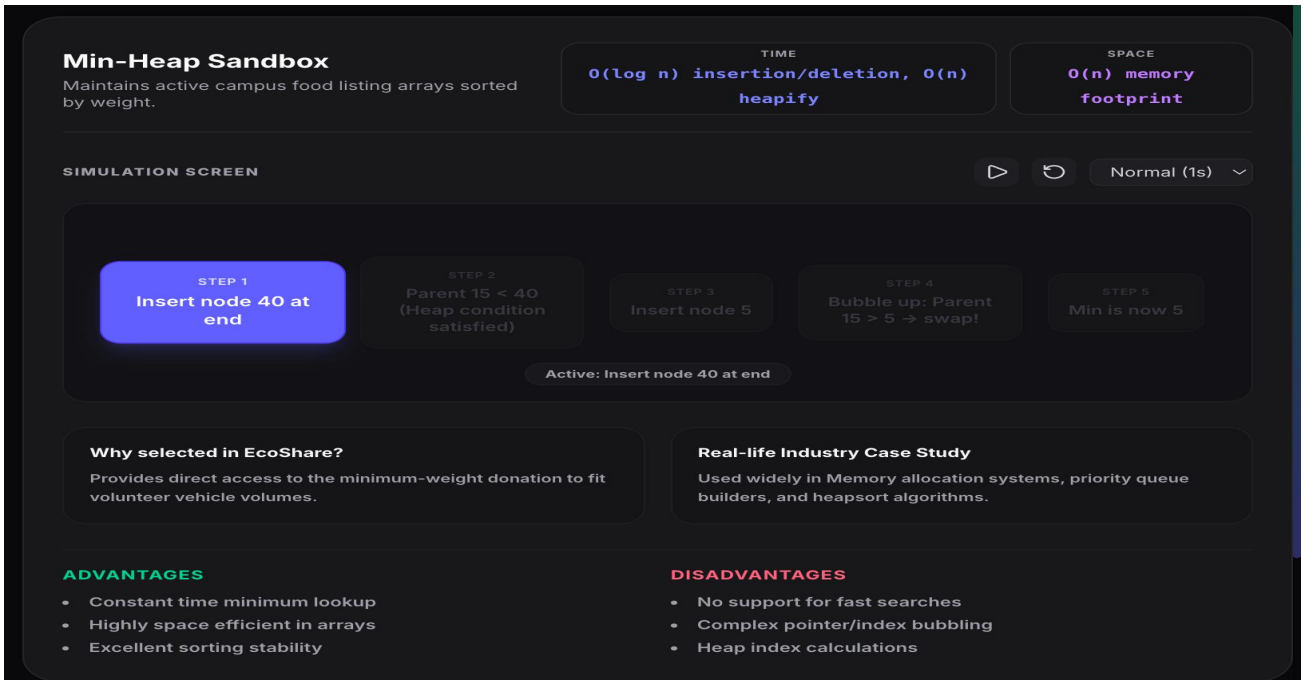
- Perishables processed first
- Dynamically adjusting dispatches
- Perfect for real-time triage

DISADVANTAGES

- Starvation risk for low priority items
- Heap index pointer adjustments
- O(n) build overhead

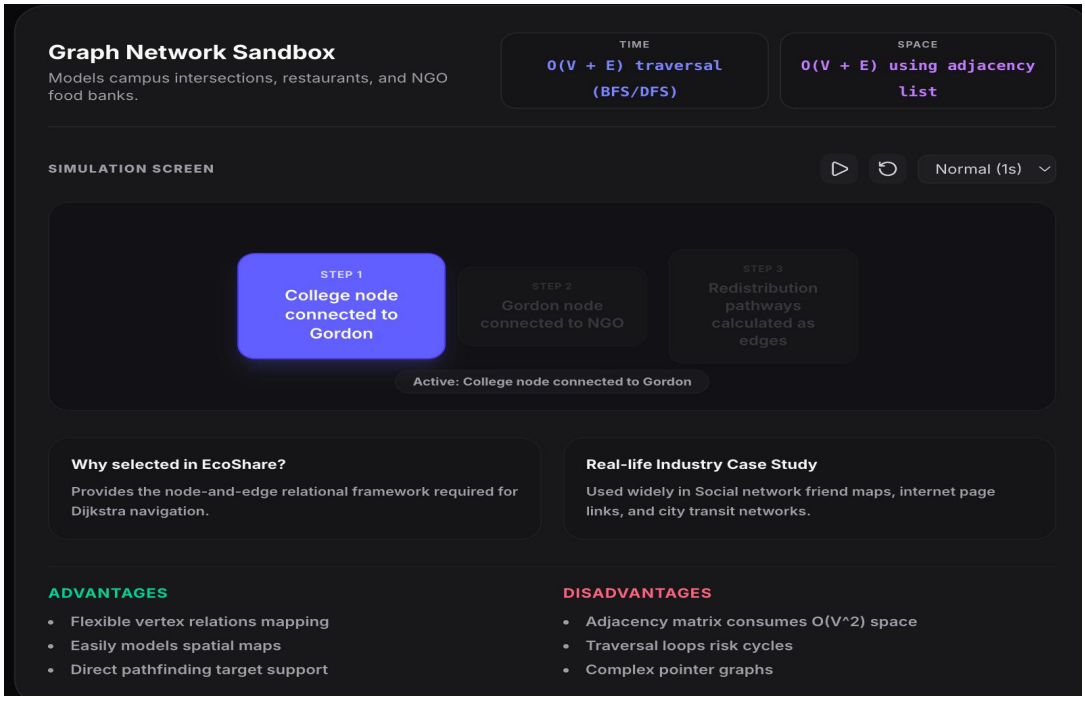
4. ***Min-Heap:***

- A Min-Heap is a complete binary tree where the smallest element is always at the root, enabling quick access to the minimum value.
- Time: O(log n) insertion/deletion, O(n) heapify
- Space: O(n)
- Use in EcoShare: Prioritizes the smallest (most urgent) food listings for efficient allocation.
- Pros: Fast minimum retrieval.
- Cons: Searching arbitrary elements is slow.



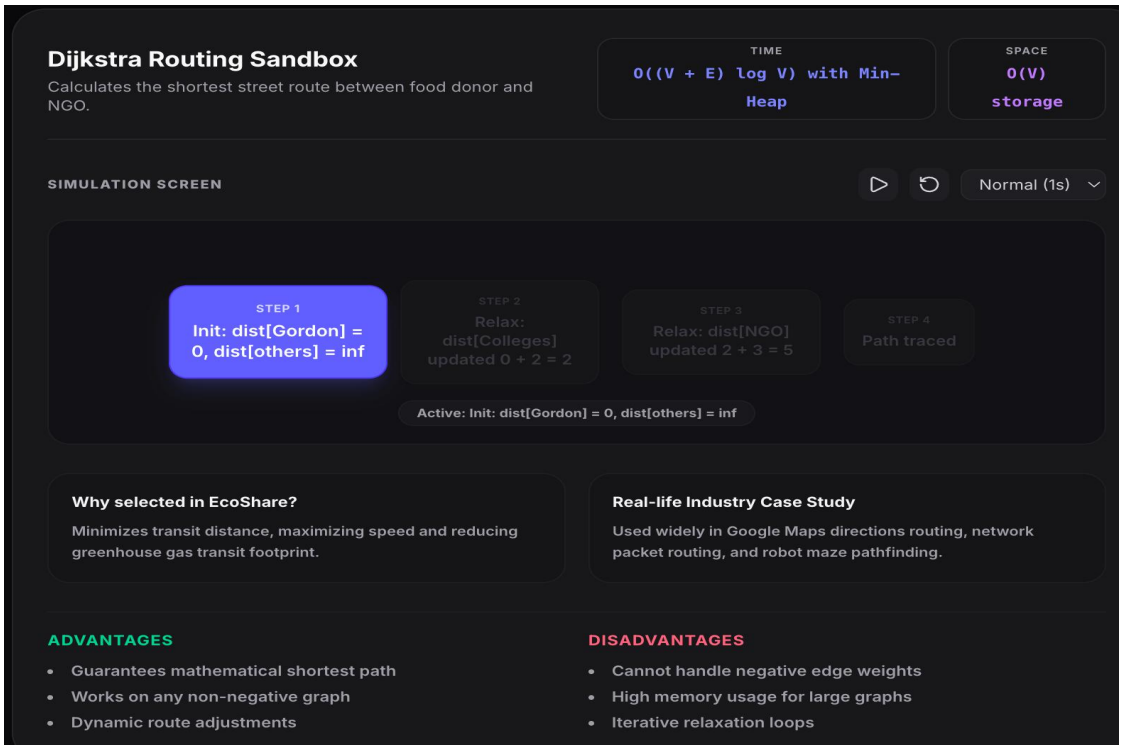
5. ***Graph:***

- A Graph is a data structure consisting of nodes (vertices) connected by edges, used to represent relationships and routes.
- Time: O(V + E) traversal (BFS/DFS)
- Space: O(V + E)
- Use in EcoShare: Models locations and finds the best delivery routes between donors and NGOs.
- Pros: Efficient for path finding and network mapping.
- Cons: Can become complex for large networks.



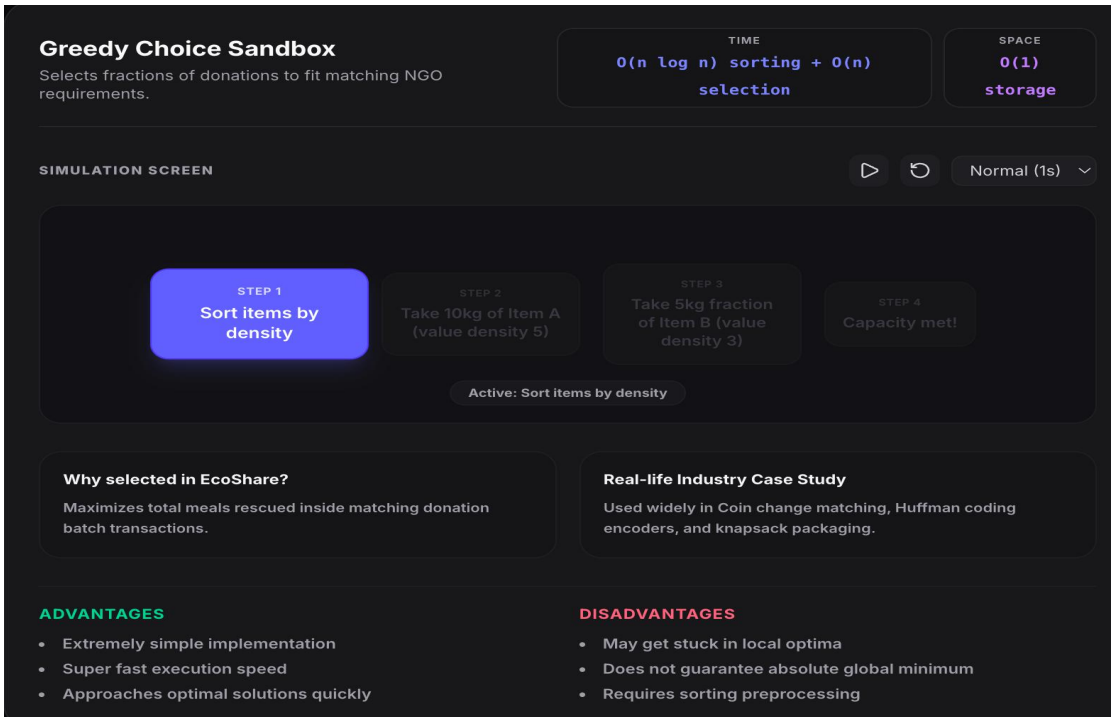
6. ***Dijkstra's Algorithm:***

- Dijkstra's Algorithm finds the shortest path from a source node to all other nodes in a weighted graph with non-negative edge weights.
- Time: O((V + E) log V)
- Space: O(V)
- Use in EcoShare: Finds the shortest delivery route between food donors and NGOs.
- Pros: Fast and accurate shortest-path calculation.
- Cons: Does not support negative edge weights.



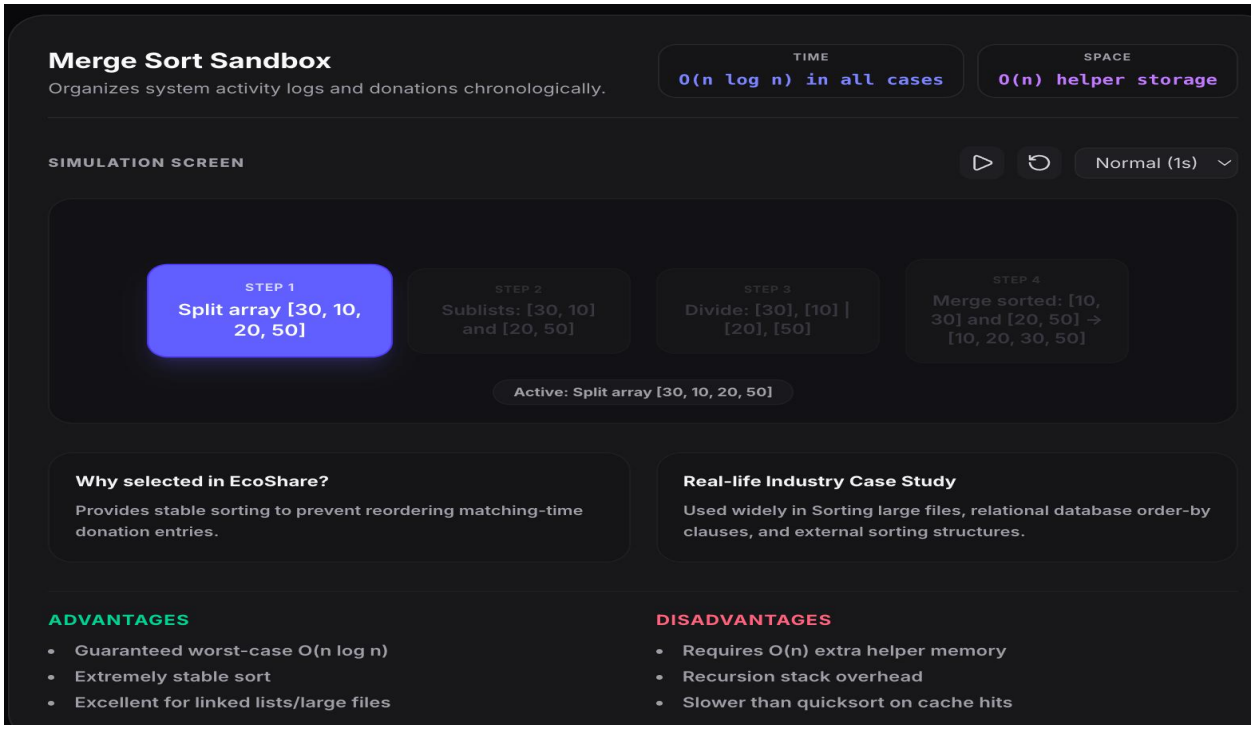
7. ***Greedy Algorithm:***

- A Greedy Algorithm makes the best local choice at each step to quickly build a solution.
- Time: O(n log n) (sorting) + O(n)
- Space: O(1)
- Use in EcoShare: Selects the best food donations first to maximize NGO requirements.
- Pros: Fast and simple.
- Cons: May not always produce the optimal solution.



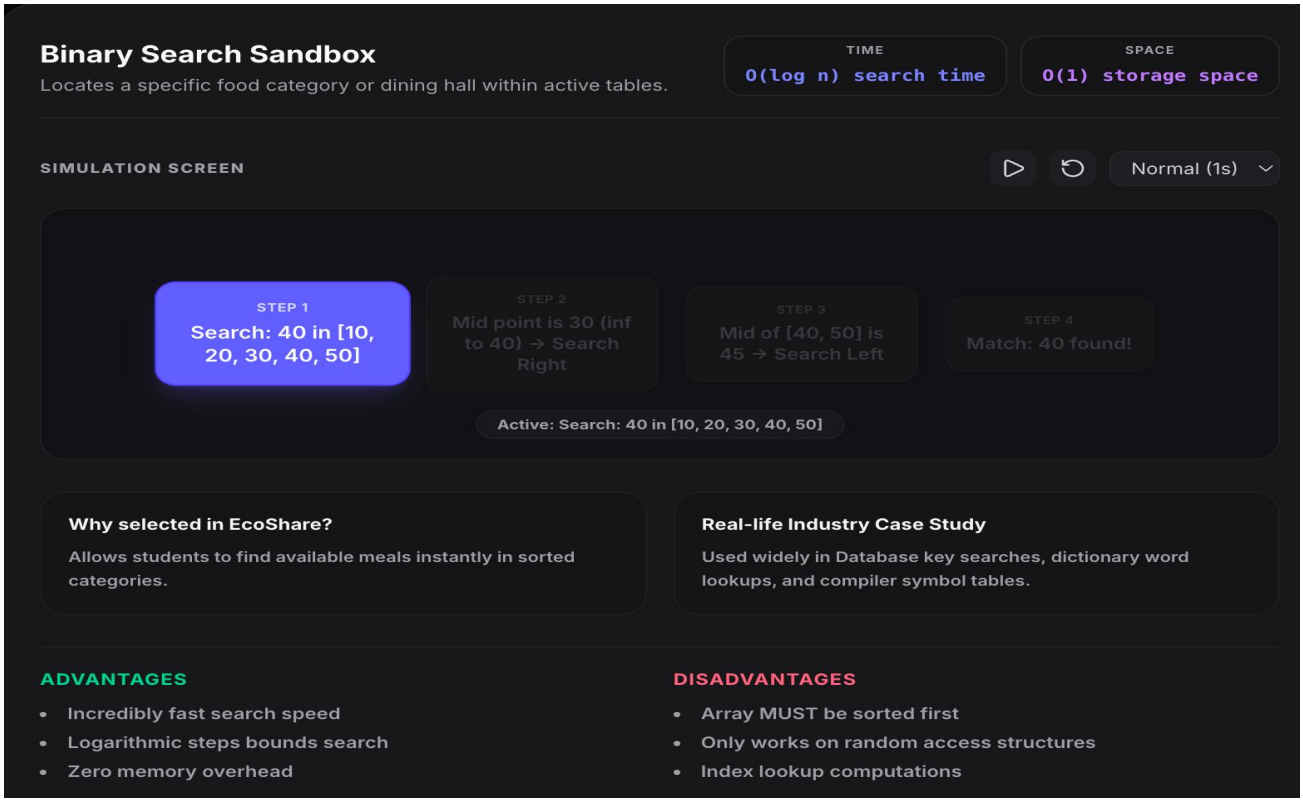
8. ***Merge Sort:***

- Merge Sort is a divide-and-conquer algorithm that recursively splits an array, sorts the parts, and merges them into a sorted array.
- Time: O(n log n)
- Space: O(n)
- Use in EcoShare: Sorts donation records and activity logs efficiently.
- Pros: Stable and consistent performance.
- Cons: Requires extra memory for merging.



9. ***Binary Search:***

- Binary Search finds an element in a sorted array by repeatedly dividing the search space in half.
- Time: O(log n)
- Space: O(1)
- Use in EcoShare: Quickly finds food categories or dining halls in sorted records.
- Pros: Very fast searching.
- Cons: Requires the data to be sorted first.



10. Quick Sort:

- Quick Sort is a divide-and-conquer algorithm that sorts data by selecting a pivot and partitioning elements around it.
- Time: O(n log n) average, O(n^2) worst case
- Space: O(log n)
- Use in EcoShare: Sorts donor records by food quantity or priority.
- Pros: Fast and memory-efficient.
- Cons: Worst-case performance depends on pivot selection.

Quick Sort Sandbox

Sorts donor kitchens by the total weight of food diverted.

TIME	SPACE
O(n log n) average, O(n^2) worst case	O(log n) recursion stack

SIMULATION SCREEN

Normal (1s)

STEP 1
Array: [30, 50, 10, 20] | Pivot: 20

STEP 2
Partition smaller: [10] | Partition larger: [30, 50]

STEP 3
Concat recursively: [10] + [20] + [30, 50] → [10, 20, 30, 50]

Active: Array: [30, 50, 10, 20] | Pivot: 20

Why selected in EcoShare?

Provides the fastest average-case array sorting to refresh active leaderboard tables.

Real-life Industry Case Study

Used widely in Standard library sort implementations, memory cache sorting, and numeric sorting processors.

ADVANTAGES

- Extremely fast in-place sorting
- Cache-friendly pointer sweeps
- No auxiliary helper array needed

DISADVANTAGES

- Unstable sort index shifting
- Worst-case O(n^2) if pivot choices fail
- High recursion depth

5. Technology Stack in Detail

The following table provides an exhaustive breakdown of the technologies utilized across the stack, justifying their selection for the EcoShare platform.

Layer	Technology	Specific Purpose in EcoShare
Frontend Core	React 19 + TypeScript	Provides a robust component architecture. TypeScript ensures strict data typing, preventing rendering errors caused by unexpected API responses.
Build Tooling	Vite	Replaces Webpack to provide near-instantaneous server starts during development and highly optimized code splitting for the production build.
Styling	Tailwind CSS v4	Allows for rapid UI development without context switching between JS and CSS files. Ensures a consistent design system and fully responsive layouts.
Animations	Framer Motion & Canvas-Confetti	Improves UX significantly by providing fluid page transitions and visual celebrations (confetti bursts) when food is successfully rescued.
3D Graphics	Three.js & React Three Fiber	Utilized on the public landing page to render interactive 3D elements (like a rotating Earth), creating a premium, modern aesthetic that captures user attention.

Layer	Technology	Specific Purpose in EcoShare
Backend Core	Node.js + Express.js	Handles concurrent API requests efficiently using an event loop. Express simplifies the creation of RESTful routes and middleware pipelines.
Database	MongoDB & Mongoose	Stores unstructured data perfectly. Mongoose provides a rigorous schema to validate data types before saving to the database.
Security Layer	bcryptjs, jsonwebtoken, Helmet	Salts and hashes passwords, securely manages stateless sessions, and automatically configures HTTP headers to defend against common web vulnerabilities.

6. Results and Expected Impact

The implementation of **EcoShare** is expected to deliver significant social, environmental, and operational benefits by improving the efficiency of food redistribution. The key expected outcomes are:

1. Reduced Food Wastage

EcoShare enables the quick redistribution of surplus food before it expires, minimizing food wastage and ensuring that edible food reaches people in need instead of being discarded.

2. Faster and Efficient Food Distribution

Using DSA algorithms such as **Priority Queue, Min Heap, Graph, and Dijkstra's Algorithm**, the platform prioritizes urgent donations and identifies the shortest delivery routes, reducing response time.

3. Positive Social Impact

The platform connects restaurants, hostels, event organizers, NGOs, and volunteers through a single digital platform, making food donation more accessible and helping support underprivileged communities.

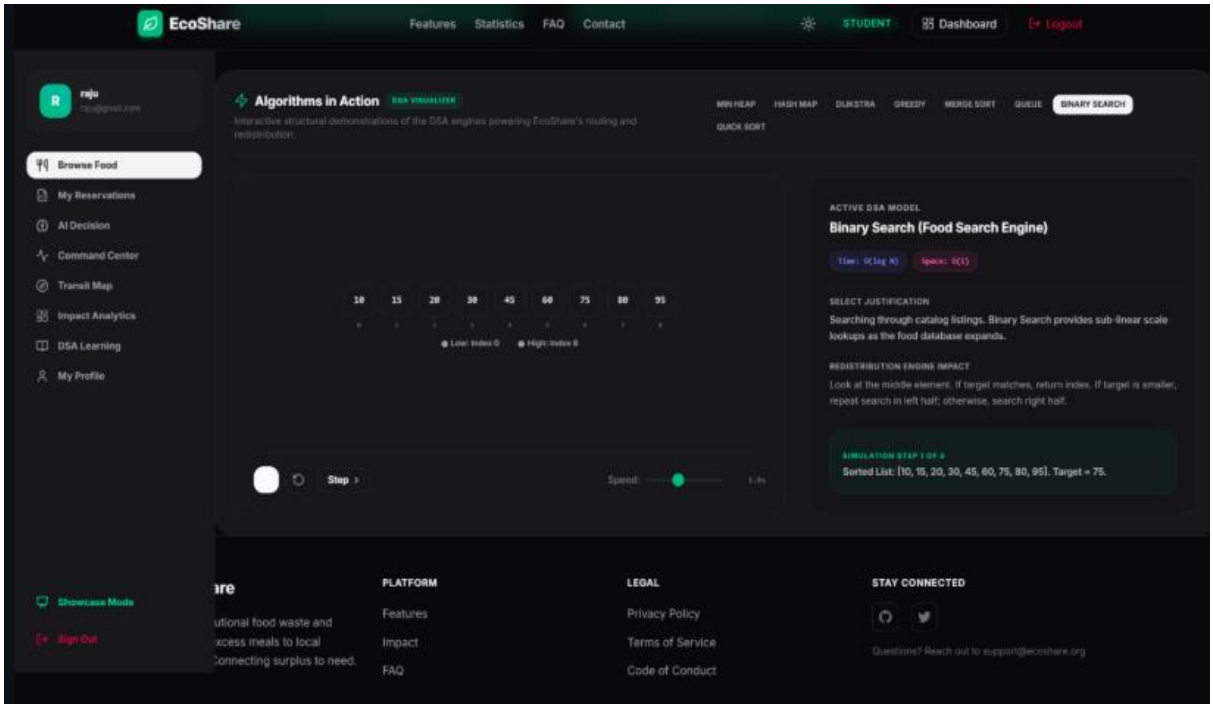
4. Environmental Sustainability

By reducing the amount of food sent to landfills, EcoShare contributes to lower greenhouse gas emissions and promotes sustainable waste management practices.

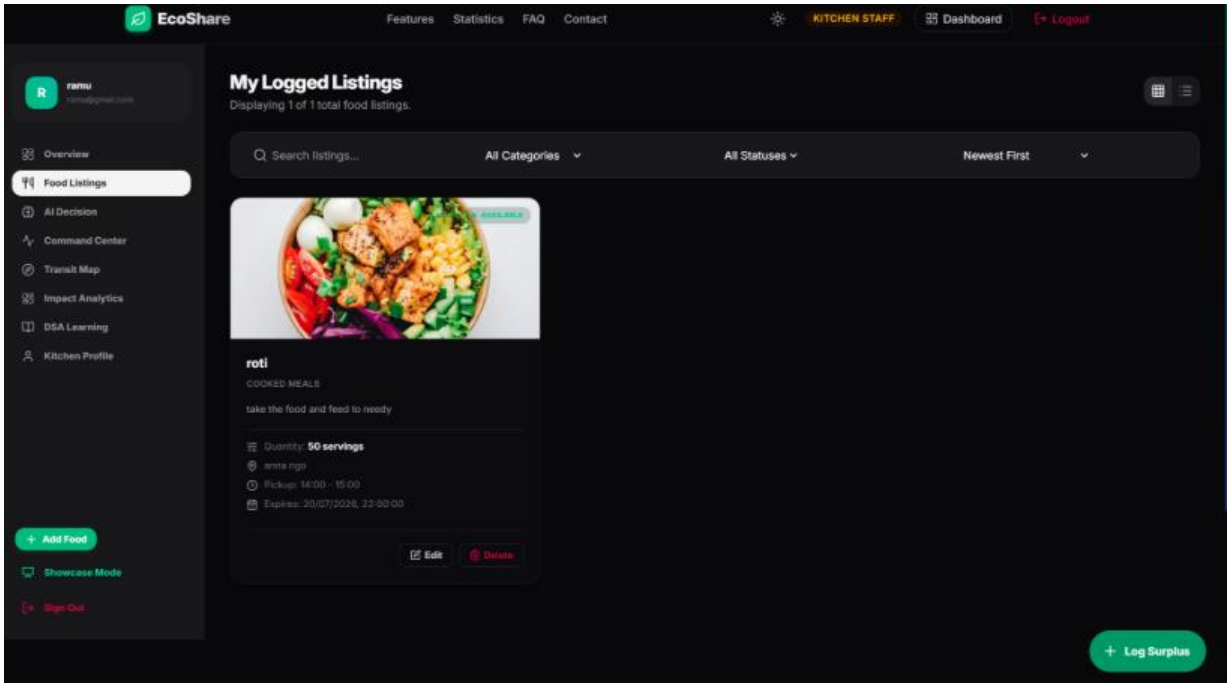
5. Improved User Experience

A responsive interface, secure authentication, real-time updates, and location-based services provide users with a seamless and efficient donation experience.

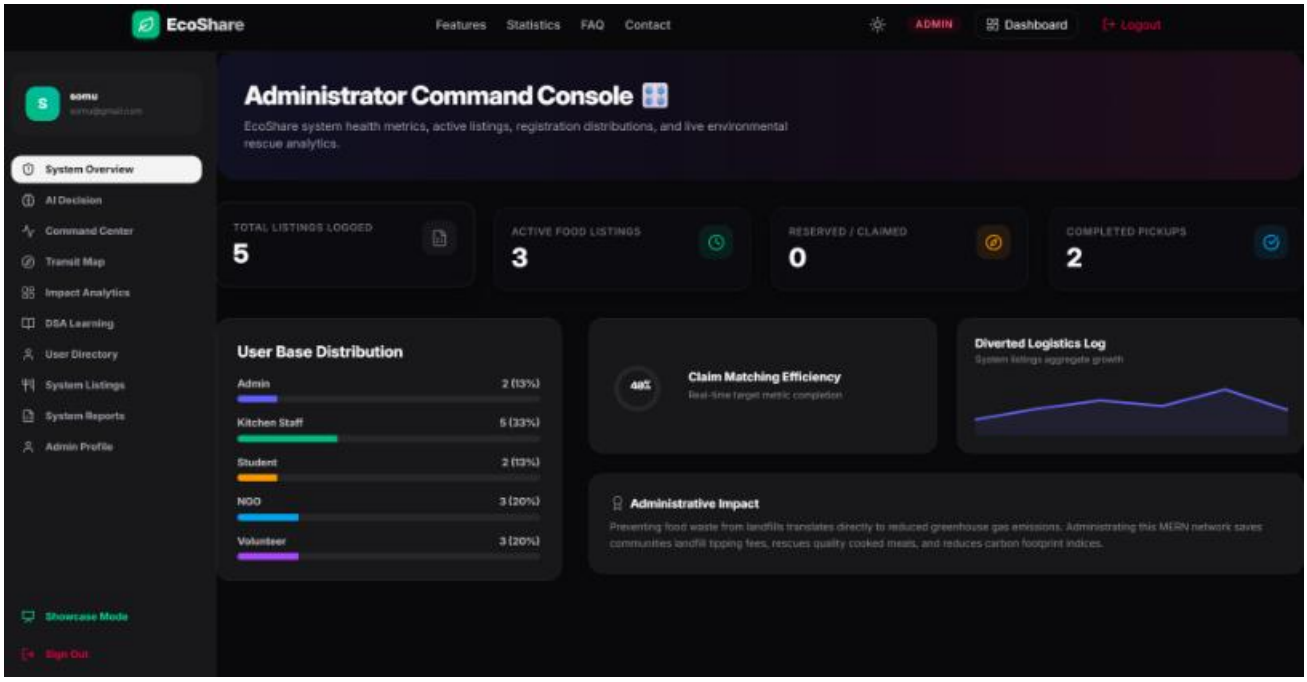
Some of the Screenshots of our website:-



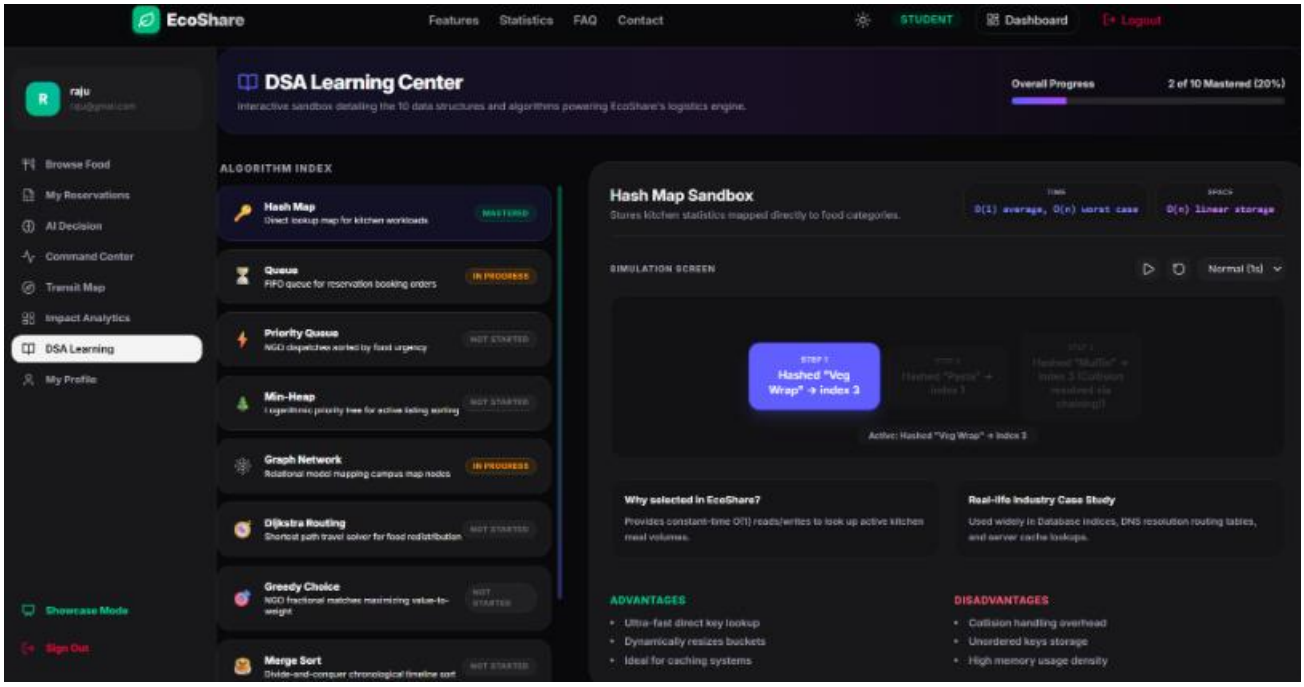
The student dashboard enables users to browse, reserve, and track surplus food through a simple and interactive interface.



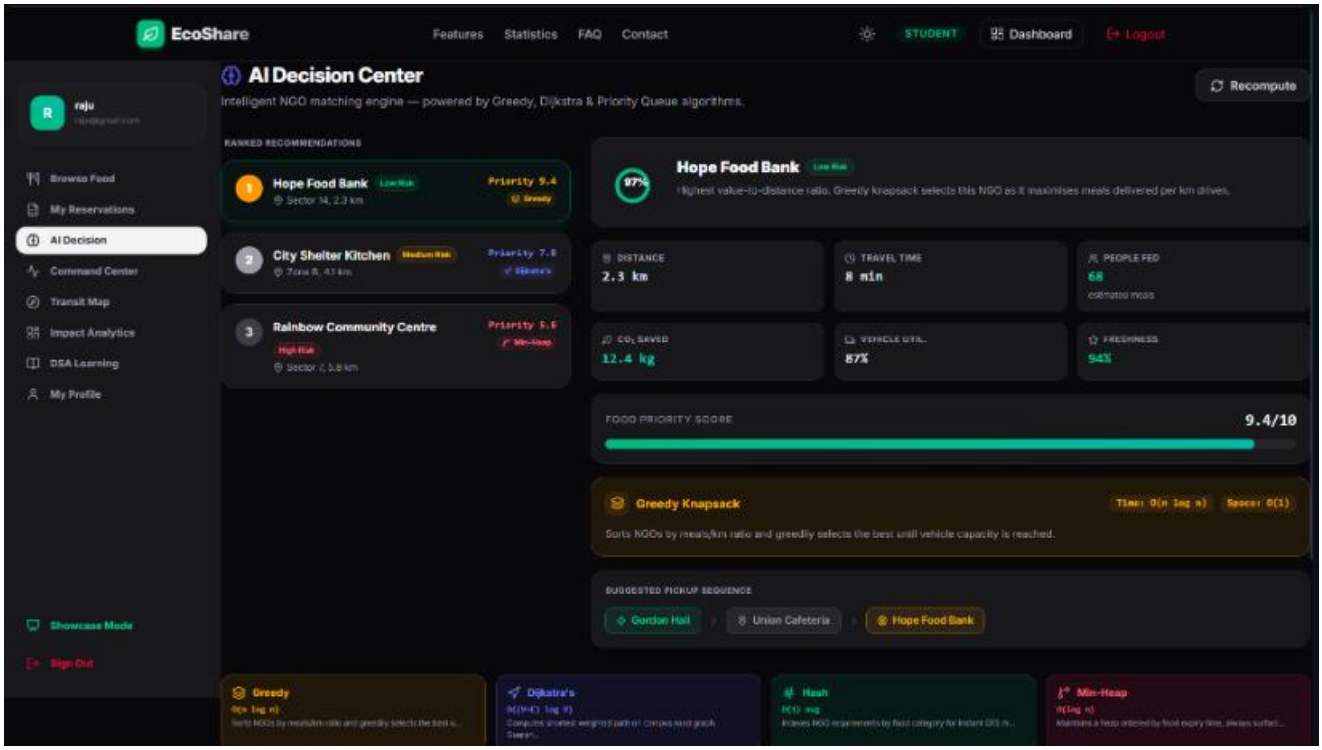
The kitchen staff dashboard allows users to log, manage, and monitor surplus food listings for efficient distribution.



The administrator dashboard provides real-time monitoring of users, food listings, system performance, and overall platform analytics.



The DSA Learning Center offers interactive simulations to help users understand algorithms used in EcoShare's food redistribution system.



The AI Decision Center intelligently recommends the best NGO and delivery route using Greedy, Dijkstra, and Priority Queue algorithms.

7.Conclusion

The **EcoShare Smart Food Waste Management System** represents a technologically advanced, highly scalable, and deeply empathetic solution to one of modern society's most pressing and paradoxical issues: the simultaneous coexistence of immense food waste alongside widespread, crippling hunger.

Through rigorous software engineering principles, EcoShare provides a frictionless, high-speed digital bridge between surplus and need. By aggressively leveraging the non-blocking speed of Node.js on the server side, the schema flexibility and geospatial indexing power of MongoDB for data persistence, and the highly interactive, component-driven capabilities of React 19 and Tailwind CSS on the client side, the platform systematically dismantled the logistical barriers that traditionally plague charitable food donation.

More than just a software application, EcoShare stands as a testament to the power of technology applied to social good. The project conclusively proves that when modern web technologies—specifically atomic database operations, real-time data synchronization, and intuitive user interface design—are directed at sociological and environmental problems, the resulting ecosystem is not only extraordinarily efficient but also highly engaging for the end user.

Ultimately, EcoShare serves as a robust, scalable blueprint for future smart city initiatives, providing a clear technological pathway toward building more sustainable, equitable, and zero-waste communities on a global scale.

6. References

The research, architectural decisions, and technical implementations detailed in this report were heavily informed by the following academic resources, official documentation, and organizational reports:

1. **MongoDB Official Documentation. (2024).** *Geospatial Queries, Indexes, and Atomic Operations*. Retrieved from https://www.mongodb.com/docs/ - Crucial for implementing the 2dsphere proximity algorithms and concurrency control.
2. **React Developer Documentation. (2024).** *React 19 Core Concepts, Hooks, and Component Architecture*. Retrieved from https://react.dev/ - Utilized for building the highly responsive frontend UI.
3. **Vite Official Guide. (2024).** *Next Generation Frontend Tooling and ES Modules*. Retrieved from https://vitejs.dev/ - Used for optimizing the build process.
4. **Tailwind CSS Documentation. (2024).** *Utility-First CSS Framework for Rapid UI Development*. Retrieved from https://tailwindcss.com/ - Foundation of the application's design system.

5. **JSON Web Tokens (JWT) Standards. (2024).** *Introduction to JSON Web Tokens and Stateless Authentication*. Retrieved from https://jwt.io/introduction - Basis for the platform's security logic.

6. **Food and Agriculture Organization of the United Nations (FAO). (2023).** *Global Food Loss and Waste: Extent, Causes and Prevention*. Comprehensive statistics regarding the global environmental impact of food waste.

7. **Framer Motion API Reference. (2024).** *Production-Ready Animation Library for React*. Retrieved from https://www.framer.com/motion/ - Used for application micro-interactions.

8. **Node.js API Documentation. (2024).** *Asynchronous Event-Driven JavaScript Runtime*. Retrieved from https://nodejs.org/en/docs/ - Foundation of the backend server.